

Chapter 1 — Revision of Class IX

Q1

Assertion: The expression `15 / 4` produces `3` when both operands are of type `int`.

Reason: Integer division gives only the integral part of the quotient.

Q2

Assertion: The expression `10 + 5 * 2` produces `20`.

Reason: Multiplication is performed before addition according to operator precedence.

Q3

Assertion: A `while` loop may execute zero times.

Reason: The condition of a `while` loop is checked before entering the loop body.

Q4

Assertion: A `do-while` loop is guaranteed to execute at least once.

Reason: Its condition is checked after the execution of the loop body.

Q5

Assertion: The `break` statement can be used to terminate a loop before its normal termination condition becomes false.

Reason: `break` transfers control to the statement immediately following the loop.

Q6

Assertion: The `continue` statement terminates the loop completely.

Reason: `continue` skips the remaining statements of the current iteration.

Q7

Assertion: In a nested loop, the inner loop completes all its iterations before the outer loop proceeds to its next iteration.

Reason: The inner loop is controlled independently within each iteration of the outer loop.

Q8

Assertion: `++x` and `x++` always produce different final values of `x`.

Reason: Both operators increase the value of `x` by one.

Chapter 2 — Classes and Objects

Q9

Assertion: An object can access an instance variable directly if the variable has suitable accessibility.

Reason: Instance variables belong to individual objects.

Q10

Assertion: Two objects of the same class can contain different values for their instance variables.

Reason: Each object has its own copy of instance variables.

Q11

Assertion: The `new` operator is used when an object is created.

Reason: It allocates memory for the object and invokes the appropriate constructor.

Q12

Assertion: A class can contain methods that operate on its data members.

Reason: Methods define the behaviour or operations associated with objects of the class.

Q13

Assertion: A static member belongs to a particular object.

Reason: A static member is associated with the class rather than individual objects.

Q14

Assertion: An instance method can directly access instance variables of the same class.

Reason: An instance method executes in the context of an object.

Q15

Assertion: A class can contain both static and non-static members.

Reason: Static members are associated with the class, whereas non-static members are associated with objects.

Q16

Assertion: Creating two objects of the same class creates two separate copies of each instance variable.

Reason: Instance variables are maintained separately for each object.

Chapter 3 — User-defined Methods

Q17

Assertion: A method declared with return type `int` must return an integer value.

Reason: The value returned by a method must be compatible with its declared return type.

Q18

Assertion: A `void` method cannot accept parameters.

Reason: Parameters and return values are two different features of a method.

Q19

Assertion: A method can be called more than once from the same program.

Reason: A method contains a reusable block of statements for performing a particular task.

Q20

Assertion: Method overloading allows methods having the same name to perform different tasks.

Reason: Overloaded methods must have different parameter lists.

Q21

Assertion: Two methods cannot be overloaded if they differ only in their return type.

Reason: The compiler distinguishes overloaded methods using their parameter lists.

Q22

Assertion: A static method can be called using the class name.

Reason: Static methods belong to the class and do not require an object for invocation.

Q23

Assertion: Actual parameters are the values supplied during a method call.

Reason: Formal parameters are declared in the method definition.

Q24

Assertion: A method can modify the value of a parameter inside the method.

Reason: Java passes primitive values to methods by value.

Q25

Assertion: A method with a return type can be used directly inside an expression.

Reason: Such a method produces a value that can participate in an expression.

Q26

Assertion: Changing the name of a method's formal parameter changes the method signature.

Reason: Parameter names are not considered while determining method overloading.

Chapter 4 — Constructors

Q27

Assertion: A constructor is automatically invoked when an object is created.

Reason: A constructor is used to initialise an object.

Q28

Assertion: A constructor can have a return type of `void`.

Reason: A constructor does not return a value.

Q29

Assertion: A class can have more than one constructor.

Reason: Constructors can be overloaded by changing their parameter lists.

Q30

Assertion: A parameterized constructor can initialise different objects with different values.

Reason: Different arguments can be supplied during different object creations.

Q31

Assertion: If a class has a parameterized constructor but no default constructor, `new ClassName()` can still be used.

Reason: Java automatically supplies a default constructor only when no constructor is explicitly declared.

Q32

Assertion: Constructors and methods can have the same name as the class.

Reason: A constructor must have the same name as its class, whereas a method need not.

Q33

Assertion: A constructor is called using the `new` operator during object creation.

Reason: The `new` operator creates an object and invokes the appropriate constructor.

Q34

Assertion: Constructors can be overloaded.

Reason: Constructors may differ in the number or type of parameters.

Chapter 5 — Library Classes and Wrapper Classes

Q35

Assertion: `Integer.parseInt("25")` produces an integer value.

Reason: `parseInt()` converts the String representation of an integer into an `int`.

Q36

Assertion: `Double.parseDouble("12.5")` produces a value of type `double`.

Reason: The method belongs to the `Double` wrapper class.

Q37

Assertion: `Character.isDigit('8')` returns `true`.

Reason: `'8'` represents a numeric digit character.

Q38

Assertion: `Character.isLetter('7')` returns `true`.

Reason: Every character represented using a character literal is considered a letter.

Q39

Assertion: Autoboxing converts a primitive value into its corresponding wrapper object automatically.

Reason: Wrapper classes allow primitive values to be represented as objects.

Q40

Assertion: Unboxing converts a wrapper object into its corresponding primitive value.

Reason: Java can automatically extract the primitive value from a wrapper object when required.

Q41

Assertion: `Integer.parseInt()` and `Integer.toString()` perform opposite types of conversion.

Reason: The former converts String to integer, while the latter converts an integer to String representation.

Q42

Assertion: `Math.sqrt(49)` returns `7.0`.

Reason: `Math.sqrt()` returns the square root as a `double`.

Q43

Assertion: `Math.pow(2, 3)` returns an integer value `8`.

Reason: `Math.pow()` returns its result as a `double`.

Q44

Assertion: A wrapper class is associated with a corresponding primitive data type.

Reason: Wrapper classes provide object representations of primitive values.

Chapter 6 — Encapsulation

Q45

Assertion: A private data member cannot be directly accessed from another class.

Reason: The `private` access specifier restricts access to the same class.

Q46

Assertion: Encapsulation allows controlled access to data.

Reason: Data can be declared private and accessed through suitable methods.

Q47

Assertion: A public method can access a private variable of the same class.

Reason: Access restrictions on a private member do not prevent its own class from using it.

Q48

Assertion: A local variable declared inside a method can be directly accessed from another method of the same class.

Reason: The scope of a local variable is limited to the block in which it is declared.

Q49

Assertion: Declaring all variables as public is an example of good encapsulation.

Reason: Public members can be directly accessed from outside the class.

Q50

Assertion: A private variable can still be modified by a public method of the same class.

Reason: Members of a class can access its private members.

Q51

Assertion: Encapsulation can help prevent accidental modification of data.

Reason: Direct access to private data from outside the class is restricted.

Q52

Assertion: The scope of an instance variable is generally greater than that of a local variable.

Reason: An instance variable belongs to an object and can be accessed by appropriate methods of the class.

Chapter 7 — Arrays

Q53

Assertion: The first element of an array is stored at index 0.

Reason: Java uses zero-based indexing for arrays.

Q54

Assertion: If an array contains 10 elements, its last valid index is 10.

Reason: Array indexing starts from zero.

Q55

Assertion: `arr.length` gives the total number of elements in a one-dimensional array.

Reason: The `length` property stores the size of the array.

Q56

Assertion: Binary search can be directly applied to an unsorted array.

Reason: Binary search repeatedly divides the search range into two parts.

Q57

Assertion: Bubble sort compares neighbouring elements.

Reason: During sorting, adjacent elements may be exchanged if they are in the wrong order.

Q58

Assertion: Selection sort repeatedly selects an appropriate element from the unsorted portion.

Reason: The selected element is placed in its correct position.

Q59

Assertion: A 2D array can be represented using rows and columns.

Reason: An element in a 2D array is accessed using two indices.

Q60

Assertion: In `a[i][j]`, `i` generally represents the row index and `j` represents the column index.

Reason: A two-dimensional array requires two indices to identify an element.

Q61

Assertion: `a.length` and `a[0].length` necessarily have the same value for every 2D array.

Reason: `a.length` represents the number of rows while `a[0].length` represents the number of columns in the first row.

Q62

Assertion: The two diagonals of a square matrix can be found using relationships between row and column indexes.

Reason: In a square matrix, the main diagonal follows `row = column`.

Q63

Assertion: Linear search can be performed on both sorted and unsorted arrays.

Reason: Linear search examines elements sequentially until the required element is found or the array ends.

Q64

Assertion: Sorting an array changes the order of its elements.

Reason: Sorting arranges elements according to a specified order.

Chapter 8 — String Handling

Q65

Assertion: `length()` returns the number of characters in a String.

Reason: A String's characters are indexed from zero.

Q66

Assertion: `charAt(0)` returns the first character of a String.

Reason: The first character of a String is stored at index `0`.

Q67

Assertion: `"Java".equals("java")` returns `true`.

Reason: `equals()` compares two Strings without considering their case.

Q68

Assertion: `"Java".equalsIgnoreCase("java")` returns `true`.

Reason: `equalsIgnoreCase()` ignores the case of characters during comparison.

Q69

Assertion: `compareTo()` can be used to arrange Strings alphabetically.

Reason: It returns a value based on the lexicographical comparison of two Strings.

Q70

Assertion: If `s1.compareTo(s2)` returns `0`, the two Strings contain the same sequence of characters.

Reason: `compareTo()` returns zero when the two Strings are equal.

Q71

Assertion: `substring(2, 5)` extracts characters at indexes 2, 3, 4 and 5.

Reason: The ending index in `substring()` is excluded.

Q72

Assertion: `indexOf()` returns the index of the first occurrence of the specified character or String.

Reason: String indexing begins from zero.

Q73

Assertion: `lastIndexOf()` is useful when we need to locate the final occurrence of a character or String.

Reason: It searches for the occurrence from the end of the String.

Q74

Assertion: `trim()` removes all spaces present between words.

Reason: `trim()` removes leading and trailing whitespace.

Q75

Assertion: `replace()` can be used to replace occurrences of a character in a String.

Reason: Strings in Java are immutable, so replacement produces a modified String value rather than changing the original String object.

Q76

Assertion: `startsWith()` can be used to check whether a String begins with a particular sequence.

Reason: It returns a boolean value.

Q77

Assertion: `endsWith()` checks whether a String ends with a specified sequence.

Reason: It compares the specified sequence with the ending portion of the String.

Q78

Assertion: Two Strings having the same characters should always be compared using `==`.

Reason: `equals()` is designed to compare the contents of Strings.

Mixed Questions

Q79

Assertion: If `arr.length` is 5, the loop

```
for(int i = 0; i < arr.length; i++)
```

accesses every valid element of the array.

Reason: The valid indexes of an array of length 5 are 0 to 4.

Q80

Assertion: A parameterized constructor can be used instead of assigning values to instance variables separately after object creation.

Reason: The constructor can receive values as arguments and assign them to instance variables during object creation.

Q81

Assertion: A method can be overloaded by changing only its return type.

Reason: Java uses the method name and parameter list to distinguish overloaded methods.

Q82

Assertion: A String can be searched using linear search even if the String array is not sorted.

Reason: Linear search examines elements one by one.

Q83

Assertion: Binary search is unsuitable for an array whose elements have not been arranged in the required order.

Reason: Binary search decides which half of the array to discard based on the ordering of elements.

Q84

Assertion: The expression

```
a[i][a.length - 1 - i]
```

can be used to access elements of the secondary diagonal of a square matrix.

Reason: For the secondary diagonal, the sum of row and column indexes remains $n - 1$.

Q85

Assertion: `Integer.parseInt("25") + "5"` produces 30.

Reason: `parseInt()` converts "25" into an integer before the addition takes place.

Q86

Assertion: If a class contains only a parameterized constructor, an object cannot be created using a constructor call without arguments.

Reason: Java does not automatically provide a default constructor when another constructor has already been explicitly declared.

Q87

Assertion: `s.substring(0, s.length())` returns the complete String `s`.

Reason: The starting index is included and the ending index is excluded.

Q88

Assertion: If `s.indexOf('x')` returns `-1`, the character `x` is not present in the String.

Reason: `indexOf()` returns `-1` when the specified character or String cannot be found.

Q89

Assertion: A private instance variable can be accessed by an instance method of the same class.

Reason: Private access restricts access from outside the class, not from within the class itself.

Q90

Assertion: In bubble sort, one complete pass can place at least one element in its final position.

Reason: The largest unsorted element can move toward the end through successive adjacent exchanges.